

Shuffle + Wazuh Integration

SOAR Automation Workflow Implementation Guide

SecureBank SOC Lab

Wazuh Manager	192.168.44.10 (SIEM)
Shuffle SOAR	Docker on SIEM or separate VM
Integration Type	Webhook (JSON alerts)
Date	May 22, 2026

Based on: "Shuffle and Wazuh integration workflow" by Ogungbade Enitan

Quick Navigation

- 1. Architecture Overview
- 2. Phase 1: Install Shuffle (Docker)
- 3. Phase 2: Create Shuffle Workflow
- 4. Phase 3: Configure Wazuh Integration
- 5. Phase 4: Fix Docker Network Issues
- 6. Phase 5: Verify Alert Flow
- 7. Troubleshooting

Architecture Overview

Integration Flow

Wazuh Agent (WEB) →

Wazuh Manager (SIEM) →

Shuffle Webhook →

Shuffle Workflow →

DFIR-IRIS / OpenCTI (future phases)

Component Roles

Component	Role	Location
Wazuh	Detects threats, generates alerts, forwards to Shuffle	SIEM VM (192.168.44.10)
Shuffle	SOAR platform - orchestrates automated responses	Docker on SIEM or separate VM
DFIR-IRIS	Incident response case management (Phase 2)	Docker
OpenCTI	Threat intelligence enrichment (Phase 2)	Docker

How It Works

1. Wazuh endpoints send logs to Wazuh Manager
2. Wazuh Manager evaluates rules → triggers alert
3. Alert forwarded to Shuffle via webhook (JSON format)
4. Shuffle workflow processes the alert
5. Shuffle creates case in DFIR-IRIS
6. Shuffle queries OpenCTI for threat intel (IP, hash)
7. All automated — no manual intervention needed

Phase 1: Install Shuffle (Docker)

Host: SIEM (192.168.44.10) or dedicated VM with Docker

Step 1.1: Install Docker (If Not Installed)

Action

```
siem@ubuntu:~$ sudo apt update
siem@ubuntu:~$ sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release
siem@ubuntu:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /u
siem@ubuntu:~$ echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] ht
siem@ubuntu:~$ sudo apt update
siem@ubuntu:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose
```

Step 1.2: Verify Docker Installation

Action

```
siem@ubuntu:~$ sudo docker --version
siem@ubuntu:~$ sudo docker-compose --version
siem@ubuntu:~$ sudo systemctl status docker
```

Expected Output

```
Docker version 24.x.x, build xxxxxx
docker-compose version 1.29.x, build xxxxxx
• docker.service - Docker Application Container Engine
  Active: active (running) ...
```

Step 1.3: Download and Configure Shuffle

Action

```
siem@ubuntu:~$ cd ~
siem@ubuntu:~$ git clone https://github.com/shuffle/shuffle.git
siem@ubuntu:~$ cd shuffle
siem@ubuntu:~/shuffle$ sudo docker-compose up -d
```

First Run Takes Time

Initial download and build can take 10-20 minutes depending on internet speed. Wait for all containers to start.

Step 1.4: Verify Shuffle Containers

Action

```
siem@ubuntu:~/shuffle$ sudo docker ps
```

Expected Containers

Container	Status	Ports
shuffle-backend	Up	5001
shuffle-frontend	Up	3001
shuffle-orborus	Up	-
shuffle-database	Up	9200

Step 1.5: Access Shuffle Web UI

Action

1. Open browser: `http://192.168.44.10:3001`
2. Default login: `admin / admin`
3. Change password on first login

If Shuffle is on Separate VM

Use that VM's IP instead of 192.168.44.10. For example, if Shuffle VM is 192.168.44.20:

```
http://192.168.44.20:3001
```

Phase 1 Checklist

- ☐ Docker installed and running
- ☐ Shuffle cloned from GitHub
- ☐ `docker-compose up -d` executed successfully
- ☐ All 4 Shuffle containers show "Up" status
- ☐ Shuffle Web UI accessible at `http://IP:3001`
- ☐ Logged in with `admin/admin` and changed password

Phase 2: Create Shuffle Workflow

Location: Shuffle Web UI at `http://192.168.44.10:3001`

Step 2.1: Create New Workflow

Action

1. In Shuffle, click "**Create Workflow**" (top right)
2. Name: `Wazuh-SOC-Automation`
3. Description: *"Automated response to Wazuh alerts"*
4. Click "**Create**"

Step 2.2: Add Webhook Trigger

Action

1. In the workflow canvas, find **Triggers** on the left panel
2. Drag "**Webhook**" onto the canvas
3. Click the webhook node to configure it

Step 2.3: Configure Webhook

Action

Setting	Value
Name	<code>wazuh_logs</code>
Call Option	Change from "hello world" to " Runtime Argument "
Variable	<code>\$exec</code>

Copy the Webhook URI

After saving, Shuffle displays the webhook URL. Copy it — you will need it for Wazuh configuration.

```
Example: http://192.168.44.10:3001/api/v1/hooks/webhook_xxxxxxx
```

Save this URL somewhere safe.

Step 2.4: Save Workflow

Action

1. Click "**Save**" (top right)
2. Click "**Start**" to activate the workflow

Do NOT Click Play Yet

Clicking the green play button now may fail because the shuffle-tools container network is not fully connected. We will fix this in Phase 4.

Phase 2 Checklist

- ☐ Workflow "Wazuh-SOC-Automation" created
- ☐ Webhook trigger added to canvas
- ☐ Webhook named "wazuh_logs"
- ☐ Call option set to "Runtime Argument" (\$exec)
- ☐ Webhook URI copied and saved
- ☐ Workflow saved and started

Phase 3: Configure Wazuh Integration

Host: Wazuh Manager / SIEM (192.168.44.10)

Step 3.1: Edit Wazuh Integration Config

Action

```
siem@ubuntu:~$ sudo nano /var/ossec/etc/ossec.conf
```

Step 3.2: Add Shuffle Integration Block

Action

Add this block **inside** the `<ossec_config>` tags (after the `</syscheck>` or `</global>` section):

```
<integration>
  <name>shuffle</name>
  <hook_url>http://192.168.44.10:3001/api/v1/hooks/webhook_YOUR_HOOK_ID</hook_url>
  <level>3</level>
  <alert_format>json</alert_format>
</integration>
```

Configuration Parameters Explained

Tag	Description	Example
<code><name></code>	Name of the integration	<code>shuffle</code>
<code><hook_url></code>	Shuffle webhook URI you copied	<code>http://192.168.44.10:3001/api/v1/hooks/...</code>
<code><level></code>	Minimum alert level to forward	<code>3</code> (forward level 3+ alerts)
<code><alert_format></code>	Format of forwarded alerts	<code>json</code>

Alert Level Reference

Level	Meaning
0-2	Info / Low (usually not forwarded)
3-5	Medium (authentication failures, policy violations)
6-7	High (attacks detected, rootkits)
8-10	Critical (multiple failures, active attacks)
11-15	Maximum severity (system compromise)

Setting `<level>3` means all alerts level 3 and above are sent to Shuffle.

Step 3.3: Save and Restart Wazuh Manager

Action

```
siem@ubuntu:~$ sudo systemctl restart wazuh-manager
siem@ubuntu:~$ sudo systemctl status wazuh-manager
```

Expected Output

```
• wazuh-manager.service - Wazuh manager
  Active: active (running) ...
  No errors in logs
```

Step 3.4: Verify Integration Config

Action

```
siem@ubuntu:~$ sudo grep -A5 "<integration>" /var/ossec/etc/ossec.conf
siem@ubuntu:~$ sudo tail -20 /var/ossec/logs/ossec.log
```

Phase 3 Checklist

- ☐ Shuffle webhook URL copied correctly
- ☐ Integration block added inside <ossec_config>
- ☐ hook_url points to correct Shuffle IP and port
- ☐ level set to 3 (or desired threshold)
- ☐ alert_format set to json
- ☐ Wazuh manager restarted successfully
- ☐ No errors in ossec.log after restart

Phase 4: Fix Docker Network Issues

Problem: Shuffle cannot connect to shuffle-tools container. This is a common Docker networking issue.

Symptom

After clicking the green play button in Shuffle, you get:

```
Connection error: shuffle-tools_1-2-0:33335/api/v1/run
```

This means the shuffle-tools container is not accessible on the Docker network.

Step 4.1: Enter Shuffle Backend Container

Action

```
siem@ubuntu:~$ sudo docker exec -it shuffle-backend sh
```

What This Does

Opens a shell inside the shuffle-backend container so you can test connectivity and install missing tools.

Step 4.2: Test Connectivity to Shuffle-Tools

Action (Inside Container)

```
/ # wget http://shuffle-tools_1-2-0:33335/api/v1/run
```

If wget Fails

This confirms the network issue. The shuffle-tools container exists but is not on the same Docker network as shuffle-backend.

Step 4.3: Exit Container and Inspect Networks

Action

```
/ # exit  
siem@ubuntu:~$ sudo docker network ls
```

Expected Networks

NETWORK ID	NAME	DRIVER	SCOPE
xxxxxxxxxx	bridge	bridge	local
xxxxxxxxxx	host	host	local
xxxxxxxxxx	none	null	local
xxxxxxxxxx	shuffle-shuffle	bridge	local <-- We need this one

Step 4.4: Inspect Shuffle Network

Action

```
siem@ubuntu:~$ sudo docker network inspect shuffle-shuffle
```

What to Look For

Check if `shuffle-tools` container is listed in the "Containers" section. If it's missing, that's the problem.

Step 4.5: Connect Shuffle-Tools to Network

Action

```
siem@ubuntu:~$ sudo docker network connect shuffle-shuffle shuffle-tools_1-2-0
```

If Container Name is Different

Docker may append random characters. Find the exact name:

```
siem@ubuntu:~$ sudo docker ps | grep shuffle-tools
```

Use the full container name from the output. Example:

```
sudo docker network connect shuffle-shuffle shuffle-tools_1-2-0.1.xpfpf6z87wklc
```

Step 4.6: Verify Connection

Action

```
siem@ubuntu:~$ sudo docker exec -it shuffle-backend sh
/ # wget http://shuffle-tools_1-2-0:33335/api/v1/run
/ # exit
```

Expected Result

wget should now succeed (download a small file or get HTTP 200). If it works, the network is fixed.

Step 4.7: Restart Shuffle

Action

```
siem@ubuntu:~$ cd ~/shuffle
siem@ubuntu:~/shuffle$ sudo docker-compose down
siem@ubuntu:~/shuffle$ sudo docker-compose up -d
```

Alternative: Restart Only Backend

```
siem@ubuntu:~$ sudo docker restart shuffle-backend  
siem@ubuntu:~$ sudo docker restart shuffle-orborus
```

Phase 4 Checklist

- ☐ Error "shuffle-tools connection failed" observed
- ☐ Entered shuffle-backend container
- ☐ wget test confirmed failure
- ☐ Docker networks listed with shuffle-shuffle present
- ☐ shuffle-tools container name identified
- ☐ docker network connect executed successfully
- ☐ wget test inside backend now succeeds
- ☐ Shuffle restarted (docker-compose down/up)
- ☐ All containers show "Up" status

Phase 5: Verify Alert Flow

Step 5.1: Trigger a Test Alert

Action (on WEB Server)

```
vmweb@ubuntu:~$ sudo apt update  
# This will trigger a sudo/auth log alert in Wazuh
```

Or trigger a higher severity alert:

```
vmweb@ubuntu:~$ sudo touch /etc/test_alert  
vmweb@ubuntu:~$ sudo rm /etc/test_alert
```

Faster Method: Use Wazuh-Logtest

```
siem@ubuntu:~$ sudo /var/ossec/bin/wazuh-logtest  
# Paste a sample log to trigger an alert
```

Step 5.2: Check Wazuh Alerts

Action (on SIEM)

```
siem@ubuntu:~$ sudo tail -50 /var/ossec/logs/alerts/alerts.log | grep shuffle
siem@ubuntu:~$ sudo grep -i "shuffle" /var/ossec/logs/ossec.log
```

Expected Output

```
wazuh-integratord: INFO: Sending alert to shuffle: ...
wazuh-integratord: INFO: shuffle response: HTTP 200
```

Step 5.3: Check Shuffle Workflow Execution

Action

1. Go to Shuffle Web UI: <http://192.168.44.10:3001>
2. Open your workflow: **Wazuh-SOC-Automation**
3. Click the **Executions** tab (or history icon)
4. Look for recent executions with green status

Expected Result

You should see workflow executions triggered by Wazuh alerts. Each execution shows:

- Timestamp of alert
- Alert level and rule ID
- Execution status: **SUCCESS**

Step 5.4: Inspect Alert Data in Shuffle

Action

1. In Shuffle workflow, click on the **webhook node**
2. View the **Execution Results**
3. Expand the JSON data to see what Wazuh sent

Expected JSON Structure

```
{
  "timestamp": "2026-05-22T...",
  "rule": {
    "level": 3,
    "description": "Successful sudo to ROOT executed.",
    "id": "5402"
  },
  "agent": {
    "id": "002",
    "name": "web01-securebank",
    "ip": "192.168.44.129"
  },
  "full_log": "May 22 ... sudo[...]: vmweb : TTY=pts/0 ...",
  ...
}
```

Phase 5 Checklist

- ☐ Test alert triggered on WEB server
- ☐ Wazuh alerts.log shows alert generated
- ☐ ossec.log shows "Sending alert to shuffle"
- ☐ Shuffle workflow shows execution triggered
- ☐ Execution status is SUCCESS
- ☐ JSON alert data visible in webhook results
- ☐ Alert contains agent IP, rule ID, and description

Troubleshooting Reference

T1: Shuffle Web UI Not Accessible

Symptom

`http://192.168.44.10:3001` returns connection refused or timeout.

Fix

1. Check if containers are running: `sudo docker ps`
2. If not running: `cd ~/shuffle && sudo docker-compose up -d`
3. Check firewall: `sudo ufw allow 3001/tcp`
4. Check port binding: `sudo ss -tlnp | grep 3001`
5. Check logs: `sudo docker logs shuffle-frontend`

T2: Wazuh Not Sending Alerts to Shuffle

Symptom

No "Sending alert to shuffle" in ossec.log. Shuffle workflow not triggered.

Fix

1. Verify integration block syntax in `/var/ossec/etc/ossec.conf`
2. Ensure hook_url is correct and reachable: `curl http://192.168.44.10:3001/api/v1/hooks/...`
3. Check alert level: Is your test alert $\geq$ level 3?
4. Check integrator daemon: `sudo ps aux | grep integratord`
5. Restart Wazuh manager: `sudo systemctl restart wazuh-manager`
6. Check integrator logs: `sudo tail -50 /var/ossec/logs/integrations.log`

T3: Shuffle Workflow Shows Failed Executions

Symptom

Workflow triggered but execution status is FAILED or ERROR.

Fix

1. Check which node failed in the workflow canvas
2. View error message in execution details
3. If shuffle-tools error: Follow Phase 4 network fix
4. If webhook error: Verify Wazuh JSON format is valid
5. Check shuffle-backend logs: `sudo docker logs shuffle-backend`

T4: Docker Network Connect Fails

Symptom

`docker network connect` returns "No such container" or "already connected".

Fix

1. Get exact container name: `sudo docker ps --format "table {{.Names}}" | grep shuffle-tools`
2. If already connected to another network, disconnect first:
`sudo docker network disconnect bridge shuffle-tools_1-2-0`
3. Then connect to shuffle-shuffle network
4. If container keeps getting recreated with new names, update docker-compose.yml to fix network assignment

T5: Alerts Sent But Shuffle Shows No Data

Symptom

Wazuh says alert sent (HTTP 200), but Shuffle webhook shows empty or malformed data.

Fix

1. Verify webhook "Call Option" is set to "Runtime Argument" with `$exec`
2. Check if webhook is expecting POST but Wazuh sends GET (rare)
3. Test webhook manually with curl:

```
curl -X POST http://192.168.44.10:3001/api/v1/hooks/... -H "Content-Type: application/json" -d
```

4. Ensure workflow is STARTED (not just saved)

T6: Shuffle Containers Keep Restarting

Symptom

`docker ps` shows containers with Restarting status or Exited.

Fix

1. Check resource limits: `free -h` and `df -h`
2. Shuffle needs minimum 4GB RAM and 20GB disk
3. Check container logs for specific errors:
`sudo docker logs shuffle-backend`
`sudo docker logs shuffle-database`
4. If database issues, clear volume and restart:
`sudo docker volume rm shuffle_shuffle-database && sudo docker-compose up -d`

Quick Command Reference

Task	Command	Host
View Shuffle containers	<code>sudo docker ps grep shuffle</code>	SIEM
Restart Shuffle	<code>cd ~/shuffle && sudo docker-compose restart</code>	SIEM
View Shuffle logs	<code>sudo docker logs -f shuffle-backend</code>	SIEM
Enter backend container	<code>sudo docker exec -it shuffle-backend sh</code>	SIEM
Restart Wazuh Manager	<code>sudo systemctl restart wazuh-manager</code>	SIEM
Check Wazuh alerts	<code>sudo tail -f /var/ossec/logs/alerts/alerts.log</code>	SIEM
Check integration logs	<code>sudo tail -f /var/ossec/logs/integrations.log</code>	SIEM

Test webhook manually	<code>curl -X POST http://IP:3001/api/v1/hooks/...</code>	SIEM
List Docker networks	<code>sudo docker network ls</code>	SIEM
Inspect network	<code>sudo docker network inspect shuffle-shuffle</code>	SIEM
Connect container to network	<code>sudo docker network connect shuffle-shuffle CONTAINER_NAME</code>	SIEM
Trigger test alert	<code>sudo touch /etc/test_alert && sudo rm /etc/test_alert</code>	WEB

Next Steps: DFIR-IRIS and OpenCTI

Future Integration Phases

Once Wazuh → Shuffle is working, extend the workflow:

1. **DFIR-IRIS Integration:** Add "DFIR-IRIS" app node in Shuffle workflow to automatically create incident cases from Wazuh alerts
2. **OpenCTI Integration:** Add "OpenCTI" app node to query threat intelligence for IPs, hashes, and domains from Wazuh alerts
3. **Email/Slack Notifications:** Add notification nodes to alert SOC analysts
4. **Auto-Response:** Add firewall/SOAR response actions (block IP, isolate host)

Shuffle + Wazuh Integration Complete

SecureBank SOC SOAR Automation

Wazuh alerts now automatically flow to Shuffle for orchestration

Next: Add DFIR-IRIS and OpenCTI nodes to the workflow